



5

AWS Services



~2s

Remediation Speed



Zero

Manual Steps



100%

Dev Coverage

AUTOMATED CLOUD COST OPTIMIZATION & REMEDIATION SYSTEM

*Event-Driven Serverless Automation using
AWS Budgets · SNS · Lambda · EC2*

AWS Budgets

Amazon SNS

AWS Lambda

EC2

CloudWatch

IAM

Adhithyan Sivaraman T

Cloud & DevOps Engineer | April 2026 | AWS ap-south-1 (Mumbai) | GitHub: github.com/Adhithyan-10

01	Executive Summary Problem, solution overview & project scope	02	Architecture Diagram End-to-end system design & service interaction
03	AWS Services Used Component breakdown and purpose	04	IAM & Security Least privilege design and role configuration
05	DRY_RUN Mechanism Safe testing vs live execution modes	06	Implementation Walkthrough Step-by-step with screenshots
07	Lambda Automation Logic Python + Boto3 function breakdown	08	Monitoring & Observability CloudWatch logs and execution traces
09	Benefits & Outcomes Key achievements and impact	10	Future Improvements Roadmap and enhancement ideas

THE PROBLEM



- ⚠ Development EC2 instances left running 24/7 during idle periods
- ⚠ No automated response when cloud budgets are breached
- ⚠ Manual monitoring leads to delayed remediation and cost overruns
- ⚠ Operational overhead of engineer-based cloud cost governance

THE SOLUTION



- ✓ Fully automated, event-driven remediation pipeline on AWS
- ✓ Budget threshold breach → SNS alert → Lambda stops dev EC2 instances
- ✓ DRY_RUN mode for safe testing before live execution
- ✓ Zero servers, zero manual intervention, zero production impact



5

AWS Services

Fully managed, serverless



~2s

Response Time

Budget breach to EC2 stop



100%

Dev Coverage

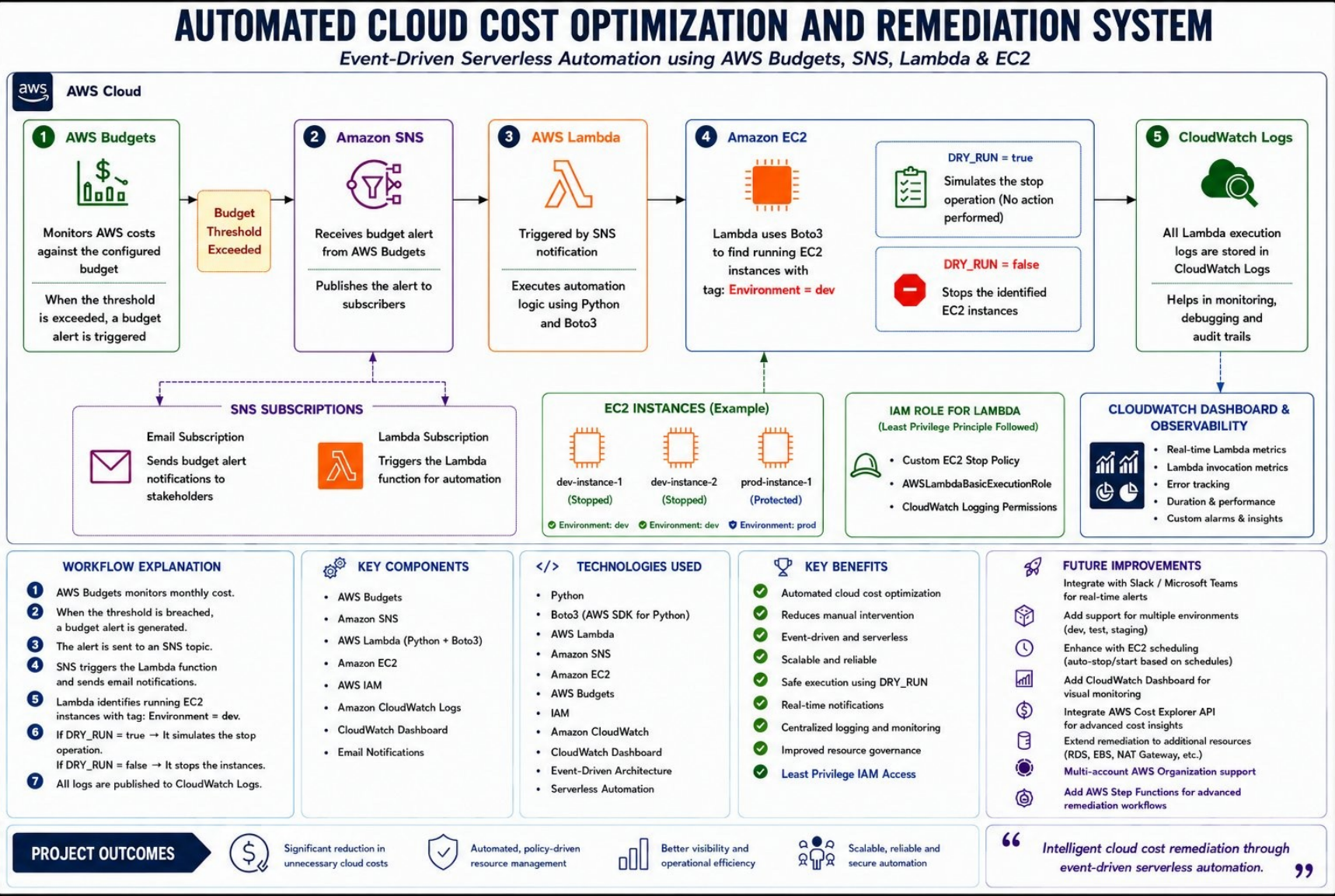
All Environment=dev instances



Zero

Manual Steps

After initial deployment



1

AWS Budgets

Monitors monthly spend. Fires alert on threshold breach.

2

Amazon SNS

Fan-out to email & Lambda simultaneously.

3

AWS Lambda

Python + Boto3. Filters dev EC2 & stops them.

4

Amazon EC2

Only Environment=dev tagged instances are stopped.

5

CloudWatch

Full execution logs & audit trail for every run.



AWS Budgets

Cost Management

Continuously monitors monthly AWS spending. Fires a budget alert event when actual or forecasted costs exceed the configured \$0.05 threshold.



Amazon SNS

Messaging & Events

Standard topic cost-alert-topic. Fan-out delivery to both the EMAIL subscription (human notification) and the LAMBDA subscription (automated remediation).



AWS Lambda

Serverless Compute

cost-optimization-lambda function in Python 3.13. Triggered by SNS. Uses Boto3 to describe EC2 instances, filter by tag, and stop dev instances.



Amazon EC2

Virtual Compute

Development instances tagged Environment=dev are the remediation targets. Production instances (Environment=prod) are completely bypassed.



CloudWatch Logs

Observability

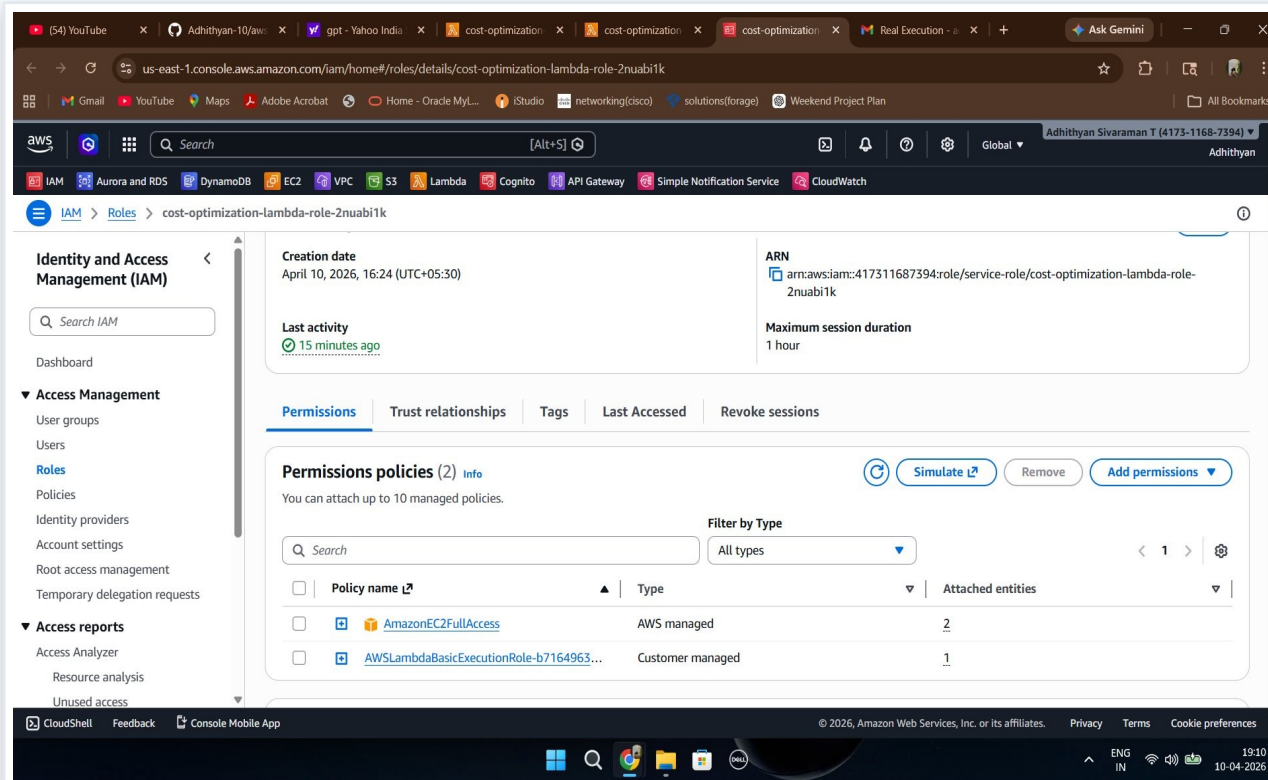
Log group /aws/lambda/cost-optimization-lambda stores all execution traces — init, alert receipt, instance discovery, stop confirmation, and performance.



AWS IAM

Security

Used AmazonEC2FullAccess during project implementation for simplicity. In production, replace with a custom least-privilege policy containing only ec2:DescribeInstances, ec2:StopInstances, and CloudWatch logging permissions.



IAM Role: cost-optimization-lambda-role-2nuabi1k | Created: April 10, 2026 | Session: 1 hour

AmazonEC2FullAccess (Implementation)

AWS Managed | Replace with Custom Policy in Production

ec2:DescribeInstances

ec2:StopInstances

EC2 read & control permissions (Production: use custom policy)

AWSLambdaBasicExecutionRole

Customer Managed

logs:CreateLogGroup

logs:CreateLogStream

logs:PutLogEvents

Used AmazonEC2FullAccess in implementation. Production best practice: custom policy with ec2:DescribeInstances + ec2:StopInstances + CloudWatch write only

DRY_RUN = true

Safe Test Mode

- SNS trigger received & processed
- EC2 describe_instances() called
- dev-tagged instances identified
- Logs: 'DRY RUN MODE — No stop'
- Logs: 'Would stop instance: i-XXX'
- ec2.stop_instances() NOT called
- Instances remain running
- Use for: pipeline validation

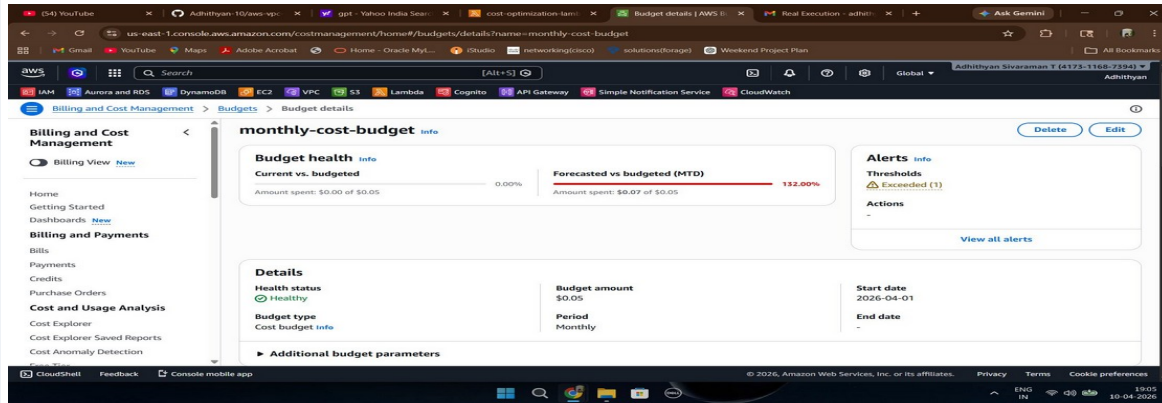
DRY_RUN = false

Live Execution

- SNS trigger received & processed
- EC2 describe_instances() called
- dev-tagged instances identified
- Logs: 'INFO: Stopping instances...'
- Logs: 'Stopped instances: [i-XXX]'
- ec2.stop_instances() IS called
- Instances transition to Stopping
- Use for: production remediation

Lambda Environment Variable | Key: DRY_RUN | Value: false (for live execution — encrypted at rest with Lambda default service key)

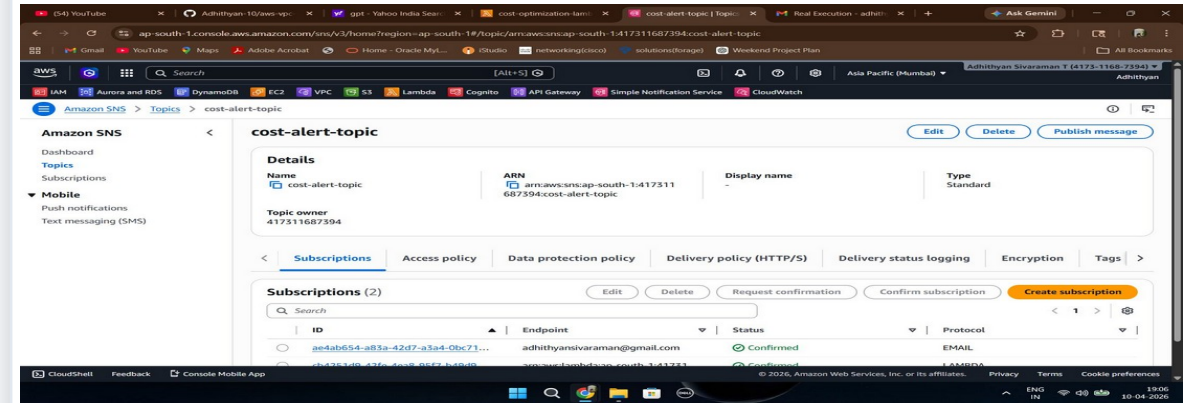
STEP 1 • AWS Budget Creation — monthly-cost-budget



Budget: \$0.05 monthly | Forecast: 132% exceeded | Alert triggered | Start: 2026-04-01

- ✓ Budget amount: \$0.05 / month (test threshold)
- ✓ Forecasted spend: 132% — triggers alert
- ✓ Budget type: Cost Budget, period: Monthly

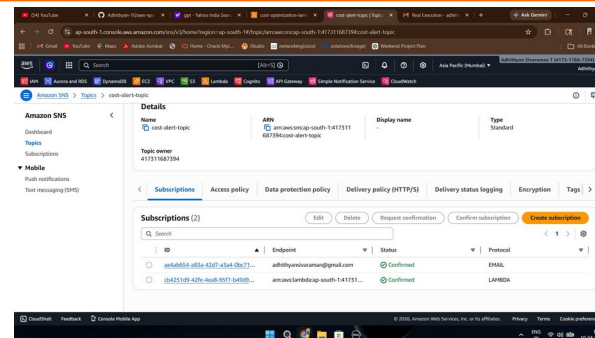
STEP 2 • SNS Topic — cost-alert-topic



ARN: arn:aws:sns:ap-south-1:417311687394:cost-alert-topic | Type: Standard | 2 Confirmed Subscriptions

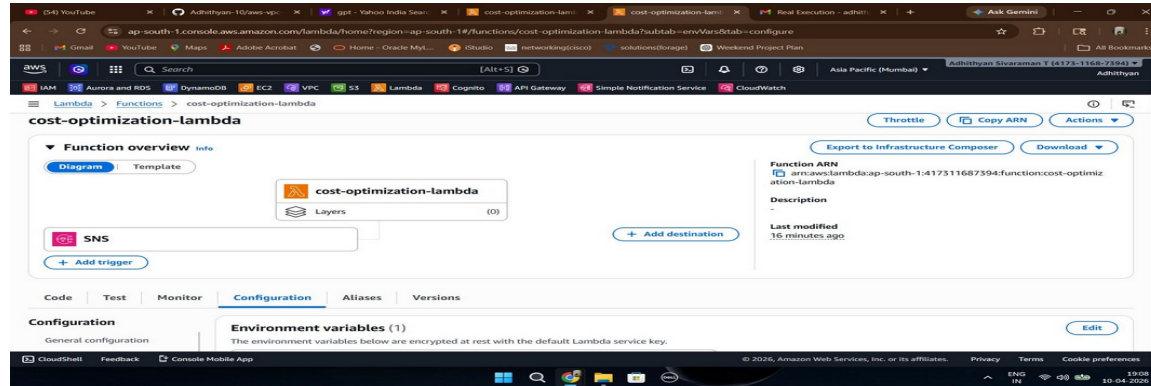
- ✓ Standard SNS topic for reliable fan-out delivery
- ✓ 2 confirmed subscriptions: EMAIL + LAMBDA
- ✓ Receives AWS Budgets alert messages automatically

STEP 3 • SNS Dual Subscriptions — EMAIL + LAMBDA (both Confirmed)



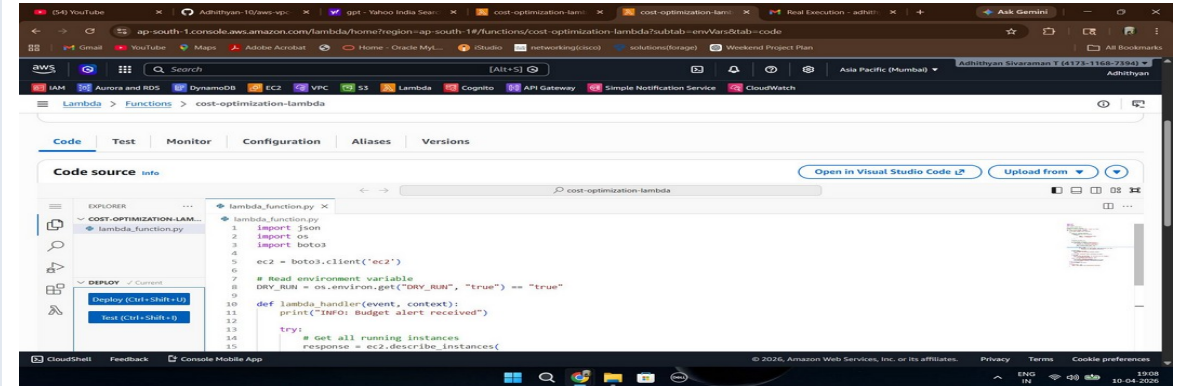
EMAIL subscription (adithyansivaraman@gmail.com) + LAMBDA subscription (cost-optimization-lambda ARN) — both status: Confirmed

STEP 4 • Lambda Function Overview



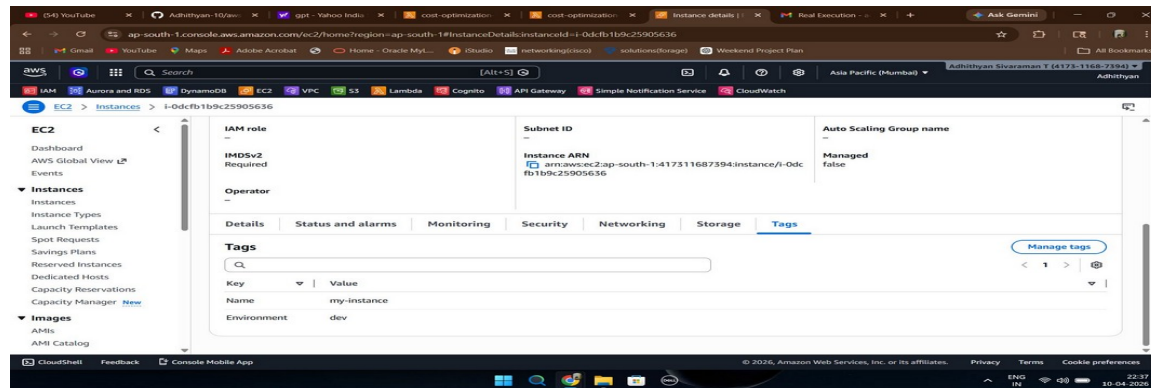
Function ARN • SNS Trigger visible • DRY_RUN env var

STEP 5 • Lambda Python Code



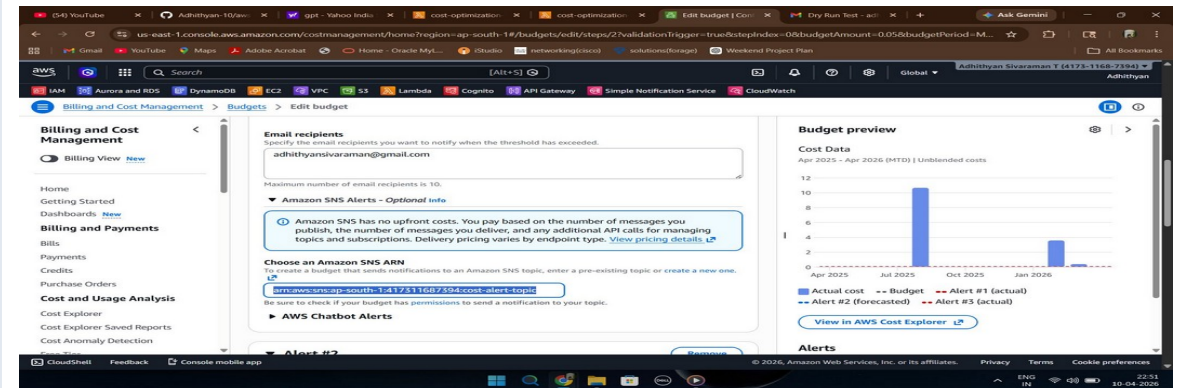
boto3.client('ec2') • DRY_RUN env • describe_instances filter

STEP 6 • EC2 Instance Tags



Tags: Name=my-instance, Environment=dev (remediation target)

STEP 7 • Budget → SNS Link



SNS ARN wired into AWS Budgets alert configuration



SNS Event Trigger



Parse Event & Log 'Budget alert received'



ec2.describe_instances(state=running)



Filter: tag Environment=dev



Check DRY_RUN flag



DRY_RUN=true → Log simulation only



DRY_RUN=false → ec2.stop_instances()



Log results to CloudWatch



Module Init

```
import json, os, boto3
ec2 = boto3.client('ec2')
DRY_RUN = os.environ.get('DRY_RUN','true') == 'true'
```



EC2 Discovery

```
response = ec2.describe_instances(
    Filters=[{'Name': 'instance-state-name',
              'Values': ['running']}])
```



Tag Filter

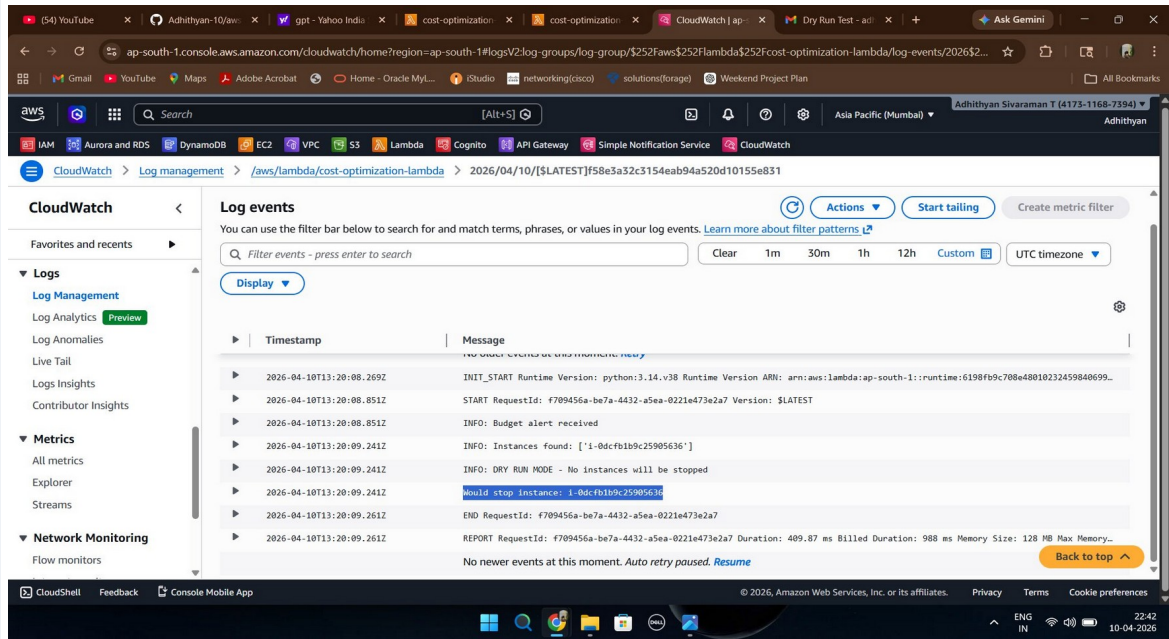
```
for r in response['Reservations']:
    for inst in r['Instances']:
        for tag in inst.get('Tags',[]):
            if tag['Key']=='Environment' and
               tag['Value']=='dev':
```



Conditional Stop

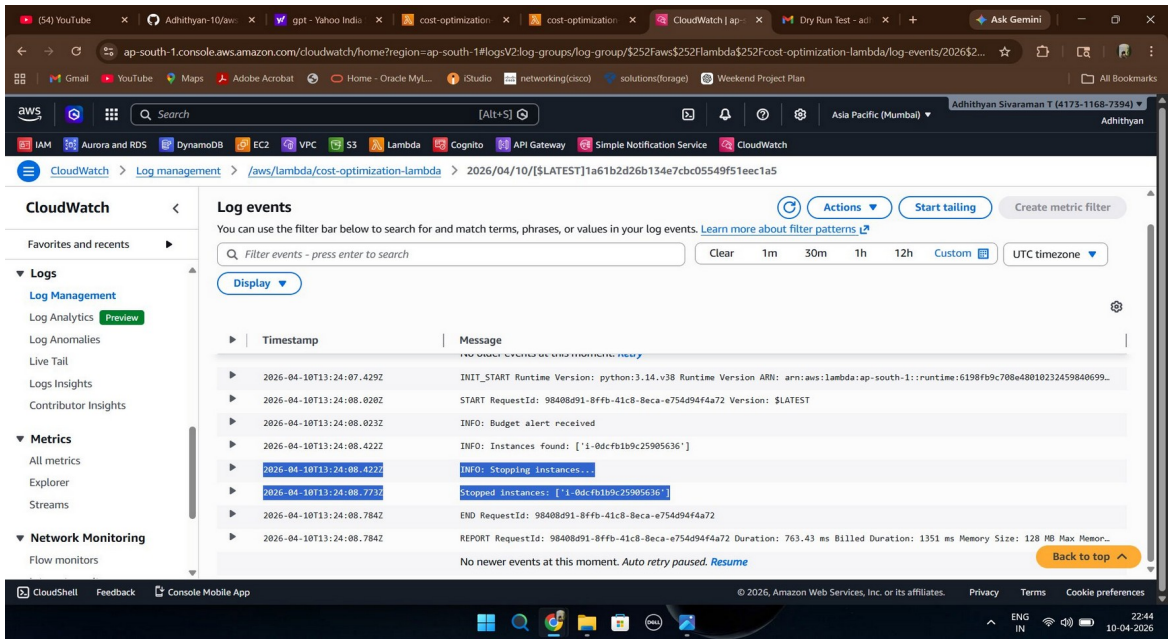
```
if DRY_RUN:
    print(f'Would stop: {ids}')
else:
    ec2.stop_instances(InstanceIds=ids)
    print(f'Stopped: {ids}')
```

DRY RUN MODE — Safe Test Execution Logs



DRY RUN KEY LOG: 'DRY RUN MODE — No instances will be stopped' + 'Would stop: i-XXX'

REAL EXECUTION — Live Remediation Logs



LIVE KEY LOG: 'INFO: Stopping instances...' + 'Stopped instances: [i-XXX]' in 763ms

CloudWatch Log Group

Log Group

/aws/lambda/cost-optimization-lambda

Log Class

Standard

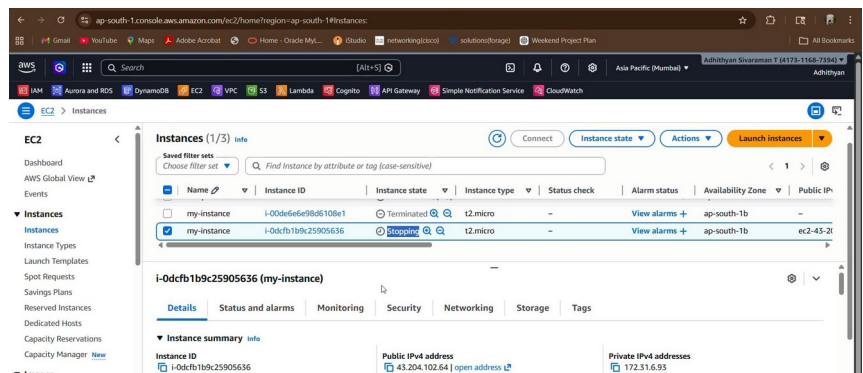
Retention

1 day

ARN

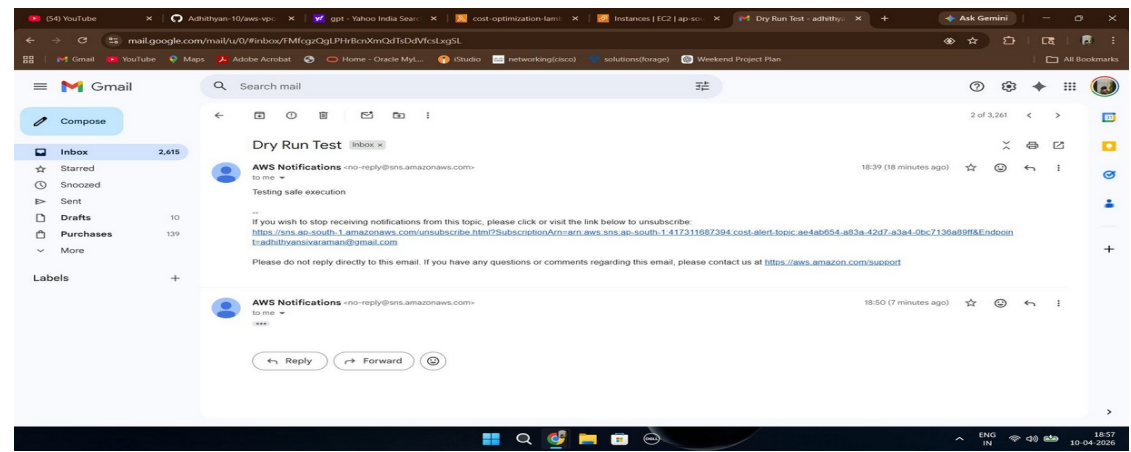
arn:aws:logs:ap-south-1:417311687394:log-group:/aws/lambda/cost-optimization-lambda:*

EC2 Instance Transitioning to STOPPING State — Live Remediation Confirmed



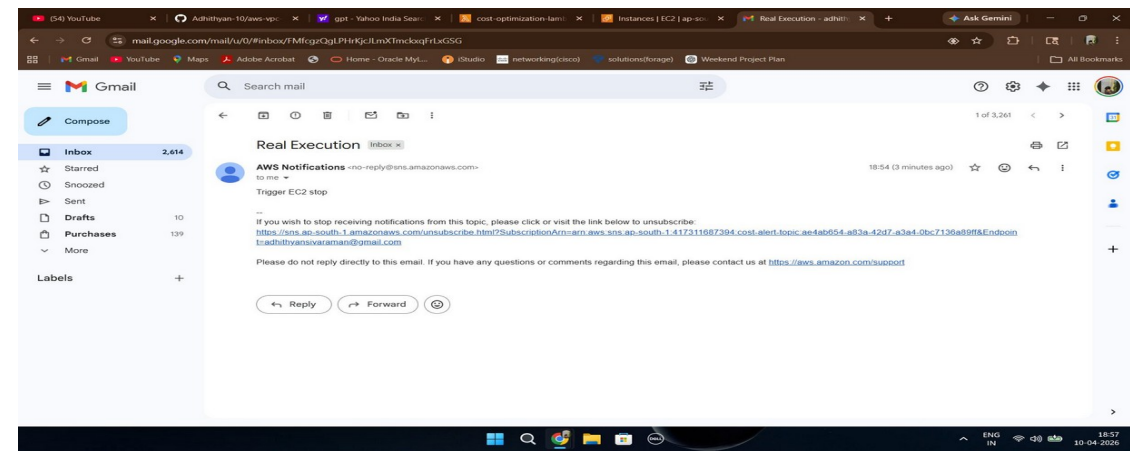
Instance i-0dcfb1b9c25905636 (my-instance, Environment=dev) | State: Stopping | Type: t2.micro | Zone: ap-south-1b

DRY RUN TEST EMAIL



Subject: 'Dry Run Test' · Body: 'Testing safe execution' · From: no-reply@sns.amazonaws.com

REAL EXECUTION EMAIL



Subject: 'Real Execution' · Body: 'Trigger EC2 stop' · Delivered: 18:54, April 10, 2026



Automated Governance

Budget thresholds enforced programmatically 24/7.
Zero manual review cycles required.



Zero Idle Dev Costs

Development EC2 instances stopped the moment a
budget breach is detected.



Real-Time Alerts

Dual notification via Email + Lambda within
seconds of threshold breach.



Serverless Scale

No infrastructure to manage. Scales to zero when
idle. Pay only per invocation.



Production Safety

Tag-based filtering ensures only Environment=dev
instances are ever stopped.



Full Observability

CloudWatch logs every action with millisecond-
precision timestamps for audit.



Least Privilege IAM

Lambda role scoped to EC2 describe/stop +
CloudWatch write only.



DRY_RUN Safety

Full pipeline validation in simulation mode before
any live execution.



CHALLENGE

IAM AccessDeniedException on Lambda invocation



SOLUTION

Attached AmazonEC2FullAccess to execution role. Verified via IAM Last Activity timestamp.



CHALLENGE

SNS email subscription not delivering



SOLUTION

Confirmed subscription by clicking the confirmation link sent to registered email. Verified 'Confirmed' status in SNS console.



CHALLENGE

Lambda trigger not receiving SNS events



SOLUTION

Created Lambda subscription explicitly in SNS (not just in Lambda console). Verified via Function overview trigger diagram.



CHALLENGE

EC2 tag filter returning zero instances



SOLUTION

Tag value 'Environment=dev' is case-sensitive. Changed instance tag from 'Dev' to 'dev' to match the Boto3 filter exactly.



CHALLENGE

Accidental live execution during testing



SOLUTION

Changed os.environ.get default to 'true' — safe mode is now the fallback even when DRY_RUN variable is absent.



CHALLENGE

Locating correct CloudWatch log stream



SOLUTION

Used RequestId from Lambda test response to find the exact log stream for targeted debugging.



Phase 1

Slack / Teams Integration

Route SNS alerts to Slack channels and Microsoft Teams via AWS Chatbot for real-time team notification with rich message formatting.



Phase 1

Multi-Environment Support

Extend tag filter to support configurable targets (dev, staging, qa) via environment variable. Tiered thresholds per environment.



Phase 2

Step Functions Orchestration

Replace single Lambda with Step Functions state machine for multi-step remediation, retry logic, and approval gates.



Phase 2

AWS Organizations Support

Cross-account IAM roles to enforce centralized cost governance across multi-account enterprise AWS environments.



Phase 3

Cost Explorer API

Query Cost Explorer from Lambda to include per-service cost breakdown in alert payloads for deeper financial context.



Phase 3

Extended Remediation

Add support for RDS instance stop/pause, EBS volume management, NAT Gateway deletion, and Elastic IP release.

CONCLUSION

Automated Cloud Cost Optimization & Remediation System

"Intelligent cloud cost remediation through event-driven serverless automation."



Event-Driven

Budget breach → SNS → Lambda → EC2 stopped.
Seconds, not hours.



Serverless

Zero servers. Zero management. Pure AWS managed services.



Production-Safe

DRY_RUN validation + tag-based filtering protect production.



Observable

Complete CloudWatch audit trail for every execution event.



Scalable

Extensible to multi-account, multi-region enterprise environments.



Secure

IAM least-privilege. Minimum permissions.
Maximum safety.